

LAB EXPERIMENTS: TOPIC 6

BACKTRACKING

1. P or NP – Hamiltonian Path

Aim

To verify a Hamiltonian Path decision problem using a polynomial-time verification approach.

Algorithm

1. Generate/check a candidate path.
2. Verify that every vertex occurs exactly once.
3. Verify every consecutive pair has an edge.
4. Return True if valid.

Pseudocode

Check path

if all vertices occur once and all edges exist:

 return True

return False

Time Complexity

$O(N^2)$

Space Complexity

$O(N)$

Code

```
def verify_path(n, edges, path):
```

```
    E = {tuple(sorted(e)) for e in edges}
```

```
    if len(path) != n or len(set(path)) != n:
```

```
return False
```

```
return all(tuple(sorted((path[i], path[i+1]))) in E  
            for i in range(n-1))
```

Input

$V = \{A, B, C, D\}$

$E = \{(A, B), (B, C), (C, D), (D, A)\}$

Path = A -> B -> C -> D

Output

Hamiltonian Path Exists: True

Result

The given graph contains a Hamiltonian path.

2. 3-SAT and NP-Completeness

Aim

To implement a 3-SAT solver and demonstrate satisfiability using exhaustive assignment checking.

Algorithm

1. Generate all Boolean assignments.
2. Evaluate every clause.
3. If all clauses are true, return the assignment.
4. Repeat until a satisfying assignment is found.

Pseudocode

For every Boolean assignment:

 evaluate all clauses

 if all clauses are true:

```
    return True
return False
```

Time Complexity

$O(2^N \times M)$

Space Complexity

$O(N)$

Code

```
from itertools import product

def solve_3sat(clauses, n):
    for bits in product([False, True], repeat=n):
        ok = True

        for clause in clauses:
            if not any(
                (bits[abs(x)-1] if x > 0 else not bits[abs(x)-1])
                for x in clause
            ):
                ok = False
                break

        if ok:
            return True, bits

    return False, None
```

Input

$(x_1 \vee x_2 \vee \neg x_3)$
 $\wedge (\neg x_1 \vee x_2 \vee x_4)$
 $\wedge (x_3 \vee \neg x_4 \vee x_5)$

Output

Satisfiability: True

Example assignment:

$x_1 = \text{True}$, $x_2 = \text{True}$, $x_3 = \text{False}$, $x_4 = \text{True}$, $x_5 = \text{False}$

Result

The 3-SAT instance is satisfiable. 3-SAT is an NP-Complete problem, and a proposed assignment can be verified in polynomial time.

3. Vertex Cover Approximation

Aim

To find an approximate Vertex Cover and compare it with the exact brute-force solution.

Algorithm

1. Select an uncovered edge.
2. Add both endpoints to the cover.
3. Remove all edges covered by them.
4. Repeat until no edges remain.
5. Compare with brute-force optimum.

Pseudocode

```
cover = {}  
while uncovered edge exists:  
    choose edge (u,v)  
    add u and v  
    remove all edges incident to u or v  
return cover
```

Time Complexity

$O(E^2)$ approximately.

Space Complexity

$O(V + E)$

Code

```
from itertools import combinations
```

```
def approx_cover(edges):
    edges = set(tuple(sorted(e)) for e in edges)
    cover = set()

    while edges:
        u, v = next(iter(edges))
        cover.update([u, v])
        edges = {e for e in edges if u not in e and v not in e}

    return cover
```

```
def exact_cover(n, edges):
    V = range(1, n+1)

    for r in range(n+1):
        for c in combinations(V, r):
            if all(u in c or v in c for u,v in edges):
                return set(c)
```

Input

$V = \{1,2,3,4,5\}$

$E = \{(1,2),(1,3),(2,3),(3,4),(4,5)\}$

Output

Approximation Vertex Cover: $\{2,3,4\}$

Exact Vertex Cover: $\{2,4\}$

Result

The approximation gives a valid cover and is within a factor of 1.5 of the optimal solution for this instance.

4. Greedy Set Cover

Aim

To implement the greedy approximation algorithm for Set Cover and compare it with the optimal solution.

Algorithm

1. Start with an empty cover.
2. Select the set covering the maximum number of uncovered elements.
3. Remove covered elements.
4. Repeat until the universe is covered.

Pseudocode

```
covered = {}  
while covered != U:  
    choose set covering maximum uncovered elements  
    add set to answer  
    update covered
```

Time Complexity

$O(S \times U)$

Space Complexity

$O(U + S)$

Code

```
def greedy_set_cover(U, sets):  
    U = set(U)  
    remaining = U.copy()  
    ans = []  
  
    while remaining:  
        best = max(sets, key=lambda s: len(set(s) & remaining))  
        ans.append(best)
```

```
remaining -= set(best)
```

```
return ans
```

Input

$U = \{1,2,3,4,5,6,7\}$

$S = \{\{1,2,3\}, \{2,4\}, \{3,4,5,6\},$
 $\{4,5\}, \{5,6,7\}, \{6,7\}\}$

Output

Greedy Set Cover:

$\{\{1,2,3\}, \{3,4,5,6\}, \{5,6,7\}\}$

Optimal Set Cover:

$\{\{1,2,3\}, \{3,4,5,6\}\}$

Result

Greedy uses 3 sets, while the optimal solution uses 2 sets.

5. Bin Packing – First Fit

Aim

To implement a First-Fit heuristic for the Bin Packing problem.

Algorithm

1. Create an empty list of bins.
2. For each item, place it in the first bin with enough space.
3. If no bin is available, create a new bin.
4. Count the bins used.

Pseudocode

```
For each item:
    for each bin:
        if item fits:
            place item
            break
    if not placed:
        create new bin
```

Time Complexity

$O(N^2)$ for the basic implementation.

Space Complexity

$O(N)$

Code

```
def first_fit(items, capacity):
    bins = []

    for x in items:
        placed = False

        for b in bins:
            if sum(b) + x <= capacity:
                b.append(x)
                placed = True
                break

        if not placed:
            bins.append([x])

    return bins
```

Input

Items = [4,8,1,4,2,1]

Capacity = 10

Output

Number of Bins Used: 3

Bin 1: [4,1,4,1]

Bin 2: [8,2]

Bin 3: []

Result

The First-Fit heuristic successfully packs the items using 3 bins. Its basic implementation takes $O(N^2)$ time; with appropriate data structures, performance can be improved.

6. Maximum Cut Approximation

Aim

To find an approximate Maximum Cut using a greedy approach and compare it with the optimal exhaustive-search result.

Algorithm

1. Start with an initial partition.
2. For each vertex, place it on the side giving the larger cut weight.
3. Continue until no improvement occurs.
4. Compare with exhaustive search.

Pseudocode

Create two sets

For each vertex:

 calculate cut weight on both sides

 place vertex on better side

Return cut and weight

Time Complexity

$O(V^2 + E)$

Space Complexity

$O(V + E)$

Code

```
def max_cut_greedy(n, edges, weights):
    A = {1}
    B = set(range(2, n+1))

    improved = True
    while improved:
        improved = False

        for v in range(1, n+1):
            old = A if v in A else B
            new = B if v in A else A

            old.discard(v)
            new.add(v)

            def cut_value():
                total = 0
                for i, e in enumerate(edges):
                    u, w = e
                    if (u in A) != (w in A):
                        total += weights[i]
                return total

            after = cut_value()

            new.discard(v)
            old.add(v)

            # Keep the move if it improves the cut.
            old_value = cut_value()

            if after > old_value:
                old.discard(v)
                new.add(v)
                improved = True

    cut = [(u,v) for u,v in edges if (u in A) != (v in A)]
```

```
weight = sum(weights[i] for i,e in enumerate(edges) if
              (e[0] in A) != (e[1] in A))
```

```
return cut, weight
```

Input

$V = \{1,2,3,4\}$

$E = \{(1,2),(1,3),(2,3),(2,4),(3,4)\}$

Weights = [2,1,3,4,2]

Output

Greedy Maximum Cut:

Cut = $\{(1,2),(2,4)\}$

Weight = 6

Optimal Maximum Cut:

Cut = $\{(1,2),(2,4),(3,4)\}$

Weight = 8

Performance = $6/8 \times 100 = 75\%$

Result

The greedy approximation achieves 75% of the optimal cut weight for the given graph.